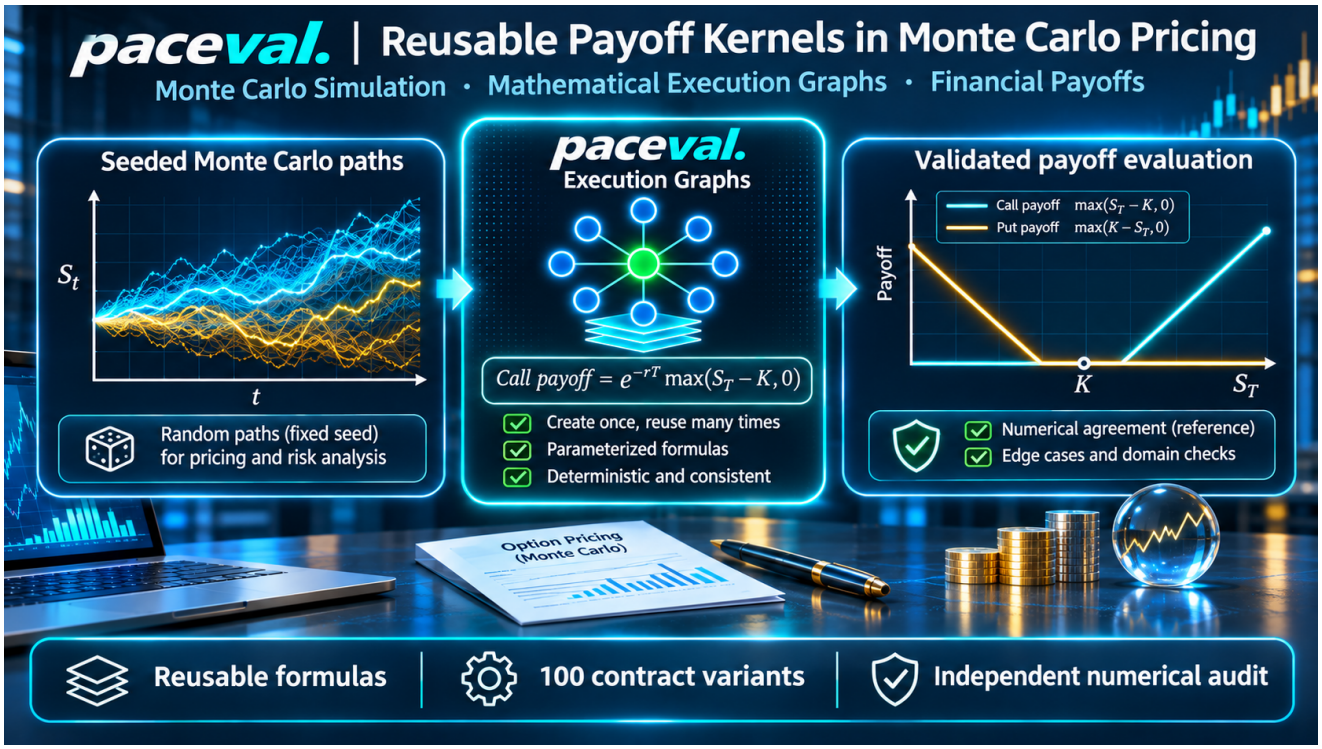
Reusable Payoff Kernels in Monte Carlo Pricing

An independently checked C++ feasibility demonstrator using reusable mathematical execution graphs with the original paceval Linux runtime

Research report | 12 November 2026 | Fully synthetic study, not financial advice



Research question. Can a reusable mathematical execution layer evaluate a broad portfolio of pre-approved financial payoff functions across fixed Monte Carlo paths while preserving numerical agreement and traceable model variants?

Executed scope	Verified result
Scenario set	1,000,000 seeded GBM paths, 500,000 antithetic pairs
Model layer	5 reusable payoff graphs, 100 configured contract variants
Payoff evaluations	100,000,000 comparisons with native C++ reference
Independent checks	300 high-precision payoffs, 40 analytic prices, 24 TINC samples

Core finding. The explicit payoff kernels have been executed successfully with paceval using the official C/C++ SDK, and every evaluated payoff matched the native reference within its predetermined tolerance. Monte Carlo pricing error is assessed independently from expression-evaluation error.

1 | Scientific purpose and architecture

Monte Carlo valuation repeatedly evaluates contract cash flows under simulated asset-price scenarios. The reusable mathematical subproblem is the explicit discounted payoff formula, not the stochastic path generator or the pricing policy. A stable API lets a single family formula receive different path features and contract parameters without rebuilding a graph for each call.

What paceval executes vs. what the C++ host owns

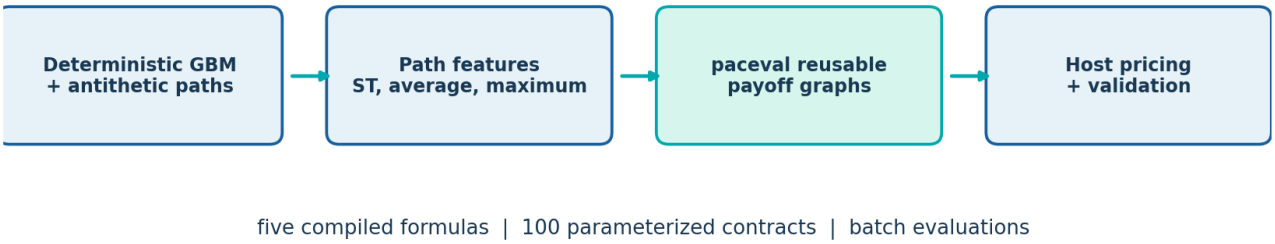


Figure 1. Exact division of responsibility: C++ generates and validates paths, whereas paceval evaluates approved discounted mathematical kernels; the C++ host performs reduction, uncertainty quantification and audit output.

This is distinct from a complete trading or risk platform. No real market data are loaded, risk-neutral inputs are fixed rather than calibrated, and the study does not test portfolio risk limits, American early exercise or transaction costs.

The five C++-created graphs are reused for 100 contract definitions; the 100 contracts are parameter sets, **not** 100 independent expression structures. This distinction matters when interpreting the graph-reuse capability.

2 | Risk-neutral scenarios and statistical design

The fixed research model is geometric Brownian motion (GBM) under a constant risk-neutral rate. With q = 0 and standard-normal Z, the maturity price satisfies:

S(T) = S(0) exp[(r - 0.5 sigma^2) T + sigma sqrt(T) Z]

Frozen research parameter	Value
Initial asset price S(0)	100
Constant risk-neutral rate r	0.03 per year
Volatility sigma	0.20 per square-root-year
Maturity	1 year
Observation steps	16 discrete monitoring dates
Random generator	mt19937_64 raw words + specified Box-Muller transform
Seed / paths	20261112 / 1,000,000 (500,000 +/- pairs)



Figure 2. Observed terminal values for paired +Z and -Z paths; paired outputs satisfy the analytical terminal-value identity under this GBM construction.

The path generator is intentionally deterministic. Both members of an antithetic pair use opposite normal innovations at each time step. The statistical standard error is estimated from **pair-average discounted payoffs**, preserving the correct independent sample count of 500,000. A Monte Carlo confidence interval concerns stochastic uncertainty, not rounding-error containment.

3 | Mathematical payoff functions

Write z+ = max(z, 0), ST = terminal asset value, A = arithmetic mean of the 16 monitored prices, M = maximum observed price including the initial spot, K = strike, and C = positive cap. Every payoff includes exp(-rT) within the paceval graph.

Payoff family	Mathematical expression (discount applied)
European call	exp(-rT) (ST - K)+
European put	exp(-rT) (K - ST)+
Capped European call	exp(-rT) [(ST - K)+ - (ST - K - C)+]
Discrete Asian call	exp(-rT) (A - K)+
Discrete lookback call	exp(-rT) (M - K)+

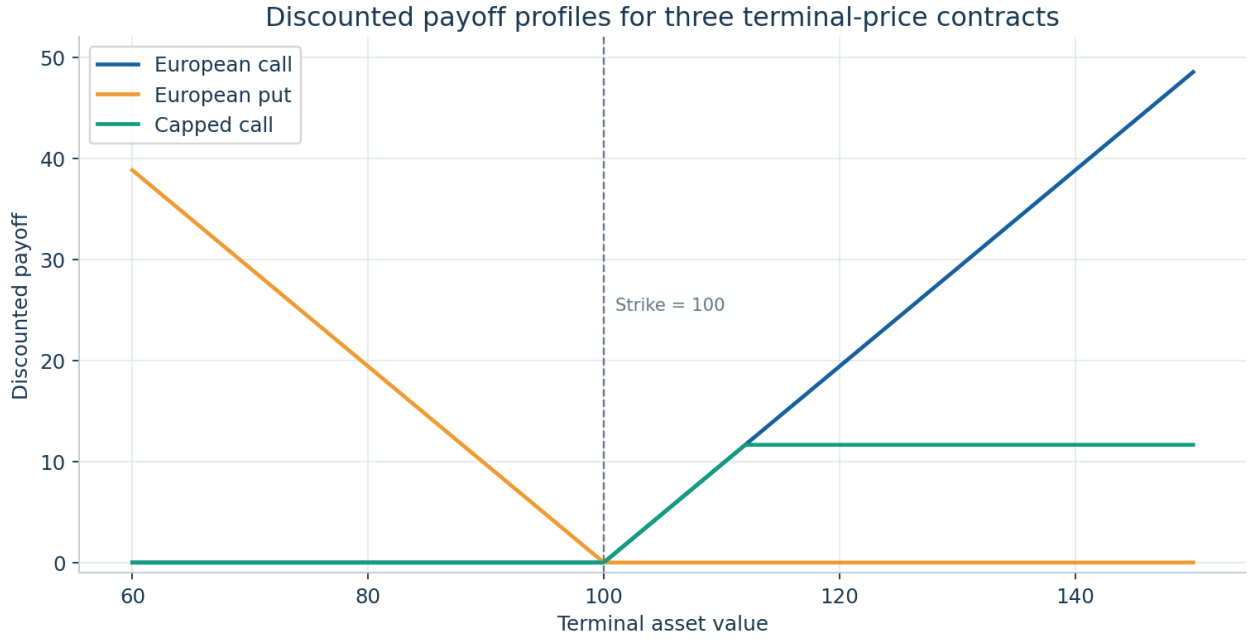


Figure 3. Terminal-price payoff profiles at strike K = 100 and cap C = 12. Asian and lookback functions depend on full-path features and are not represented by this terminal-only cross-section.

The verified paceval parser uses **(z + abs(z))/2** to express a positive part. The C++ source preserves the exact formula strings in a fixed list. Only explicitly approved formulas are evaluated; input validation rejects nonfinite values before calling the engine. An Asian average and lookback maximum are computed by the C++ host; these two features represent discrete monitoring, not continuous-exercise assumptions.

4 | SDK integration and contract governance

The project directly compiles the official **paceval_main.cpp** SDK interface and includes **paceval_main.h** from the application. An architecture-compatible proprietary Linux runtime is supplied externally at build time. The application does not implement an alternative dynamic-function-dispatch adapter.

paceval API call	Actual research use
paceval_InitializeLibrary / FreeLibrary	Define and close the mathematical engine lifetime
paceval_CreateComputation	Create five reusable payoff graphs plus one TINC graph
paceval_dGetComputationResultExt	Evaluate payoff batches for fixed formula handles
paceval_dGetComputationResult	Read point result and TINC interval for 24 discounting samples
paceval_GetIsError / DeleteComputation	Check graph status and release handles

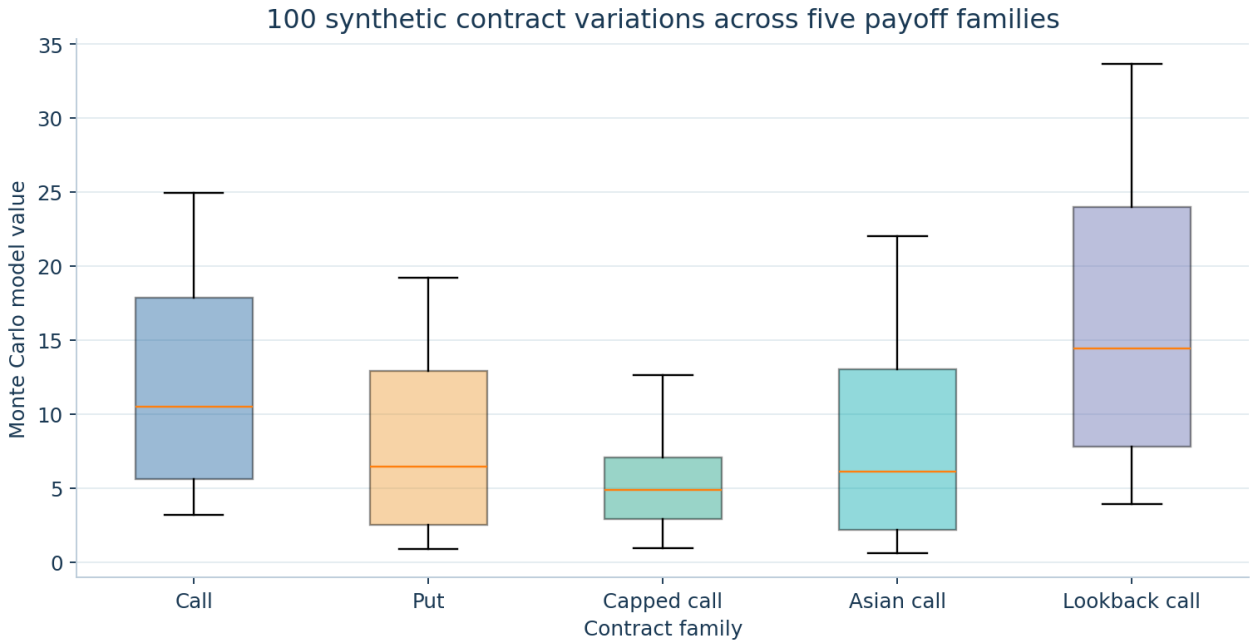


Figure 4. Published 100-contract output grouped by formula family. Financial values are hypothetical and model-dependent.

A strike-version update is deliberately restricted to a single contract. The revised cash flow is evaluated with the same approved graph handle, and a separate unaffected contract is reevaluated before and after the modification. The C++ host owns version identity, strike selection and validation; no model identity is inferred from a handle alone.

5 | Full-scale results and analytic controls

Verified numerical quantity	Observed result
Path/contract evaluations	1,000,000 paths x 100 contract variants = 100,000,000
Native C++ payoff comparisons	100,000,000 within tolerance
Number of analytic European contracts	40 (20 calls, 20 puts)
Largest standardized analytic deviation	1.406 antithetic-pair standard errors
100-decimal independent payoff checks	300 / 300 within tolerance
Observed pointwise TINC enclosures	24 / 24 (smooth discounted-terminal kernel)

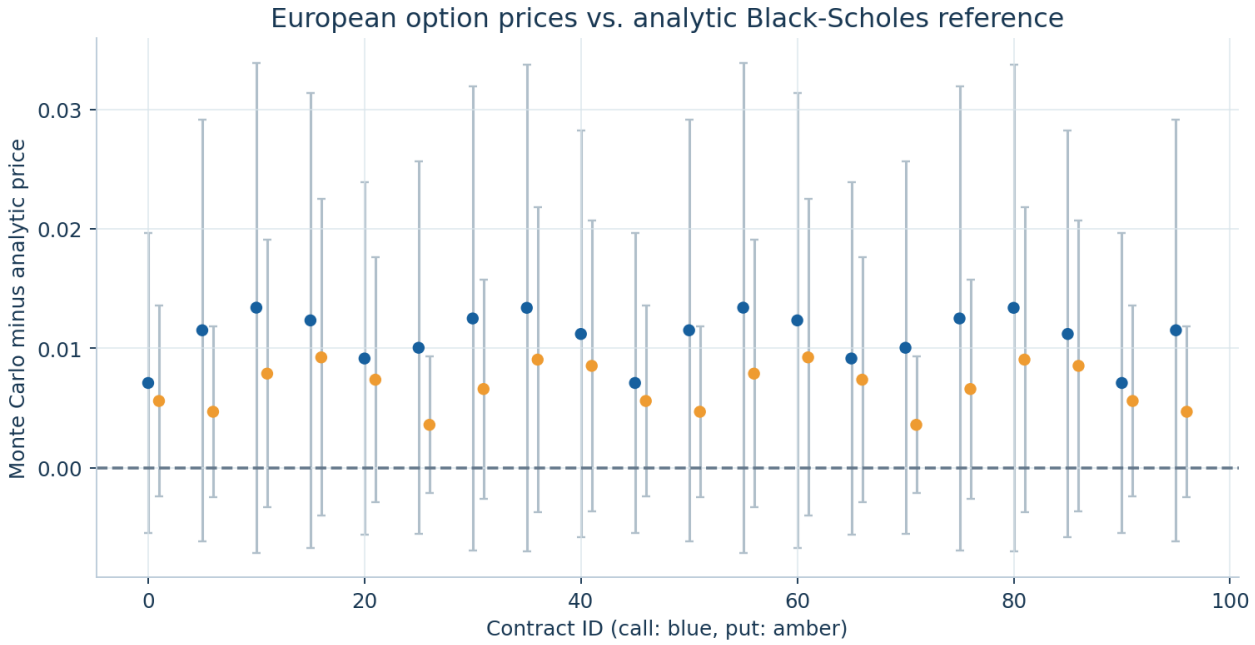


Figure 5. For European call and put variants, the Monte Carlo price minus the analytic Black-Scholes reference; error bars are +/-1.96 estimated antithetic-pair standard errors.

The native C++ mathematical kernel matches paceval pointwise across the full workload. The Monte Carlo price is not required to equal the analytical expectation exactly because the 1,000,000 pseudo-random paths introduce sampling uncertainty. All 40 European prices meet the predeclared permissive gross-error criterion |MC - analytic| <= 6 SE + 0.025. This criterion does not establish calibration to any traded security.

6 | Independent numerical precision checks

The independent Python auditor uses mpmath at **100 decimal digits**. Published binary64 input values are promoted into high precision before the exact payoff formula is reevaluated; the script does not call paceval, and it independently recomputes the Black-Scholes reference for European options.

Check	Independent result
Selected discounted payoff outputs	300, no tolerance failures
Largest absolute 100-digit payoff discrepancy	3.327e-15
Analytic Black-Scholes price references	40, zero failures
Expected antithetic pair terminal identity	64 pairs, zero failures
Model revision samples	16, native reference and unaffected-contract check

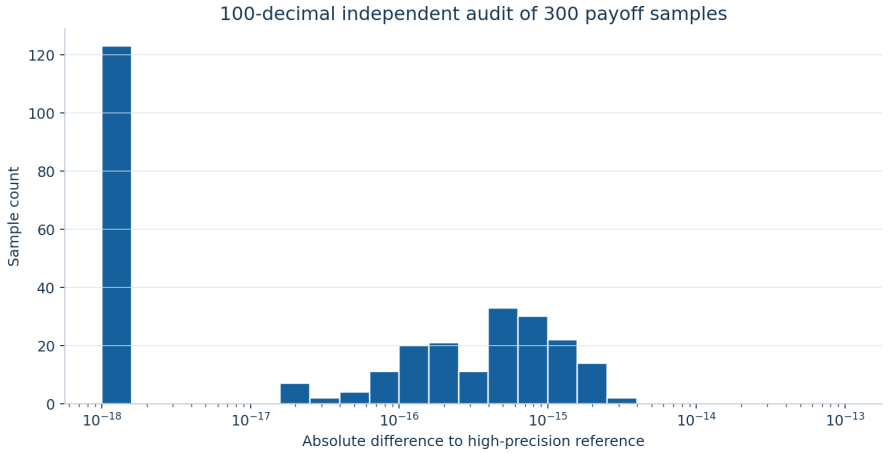


Figure 6. Absolute deviations from the independently calculated 100-decimal payoff results. The histogram is sample-based and does not assert a universal floating-point error bound.

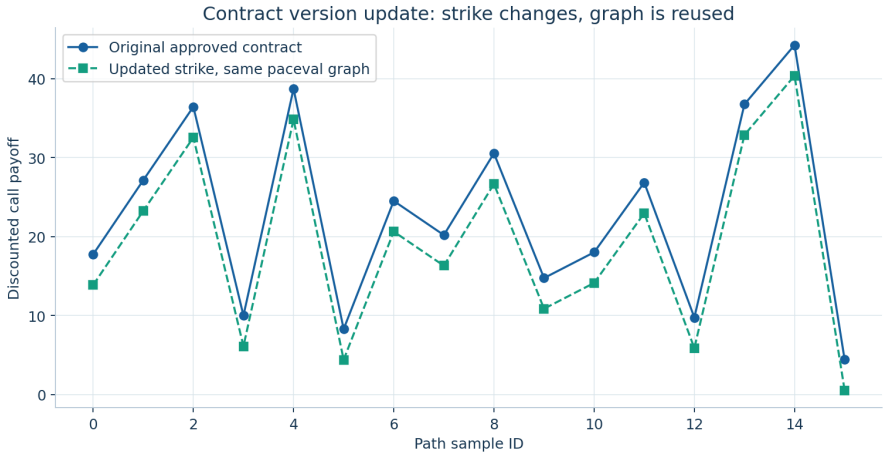


Figure 7. A deliberately changed strike is processed by the original reusable formula graph without changing a separate contract.

7 | Interval experiments and scientific limits

The discrete call/put payoff contains a kink at the strike. In the tested paceval build, requesting TINC on the non-smooth positive-part call formula produced an invalid interval status at selected points; the research script therefore conducts its separate 24-sample TINC check for the smooth financial quantity **ST exp(-rT)**. This distinction is explicit so that an interval for the discounted underlying is not mistaken for an independently certified option-price interval.

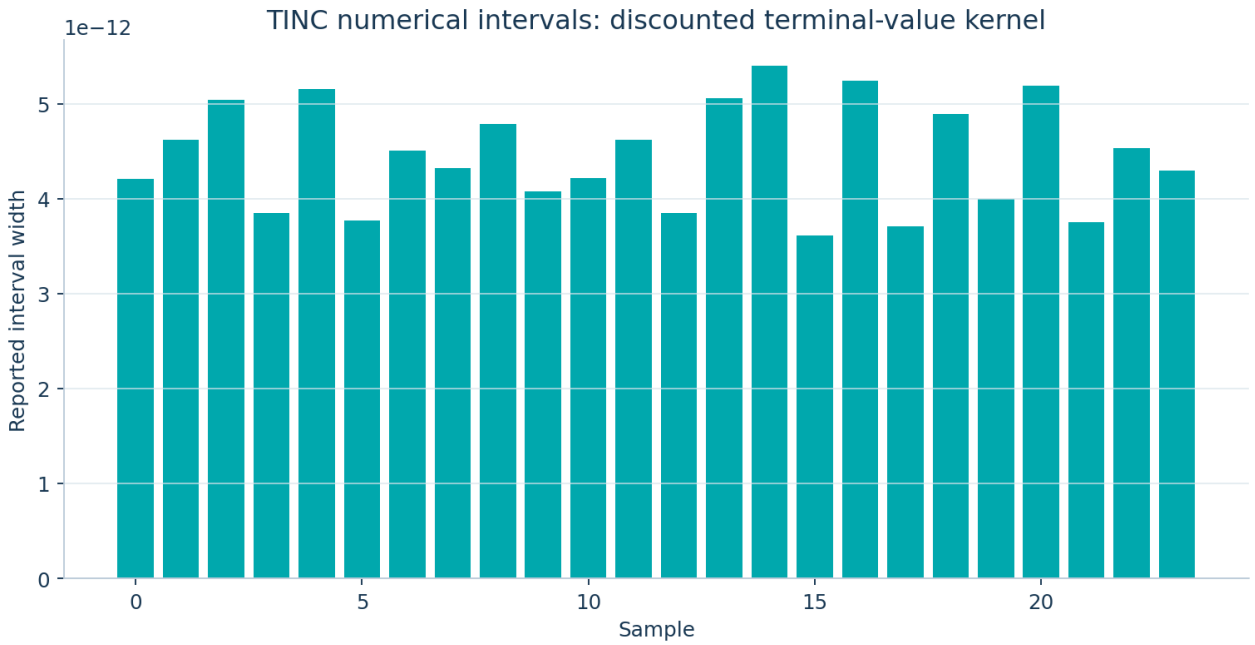


Figure 8. Measured reported interval widths for 24 smooth discounted-terminal samples. All 24 reference values fall inside their reported intervals in this fixed audit.

The positive TINC results are **sample-wise observations**, not a global theorem of outward rounding or validated pricing intervals. The source code, reference inputs and high-precision audit are published in the project. The model also excludes market parameter uncertainty, discretization corrections for continuous extrema, stochastic model risk, correlations, counterparty default and external financial advice.

Independent hardware evidence is restricted to Linux x86-64 using the supplied original runtime. ARM support, parallel graph scheduling, FPGA/ASIC execution, deterministic execution deadlines and power efficiency are outside the measured scope. No speed or energy advantage is claimed by this feasibility study.

8 | Reproducibility and scientific references

Build from repository root (Linux):

```
export PACEVAL_LIB=/absolute/path/to/libpaceval_linux_sharedLIB.so
cmake -S src -B build -DCMAKE_BUILD_TYPE=Release -DPACEVAL_RUNTIME="$PACEVAL_LIB"
cmake --build build -j
ctest --test-dir build --output-on-failure
```

Reproduce the full executed workload:

```
./build/monte_carlo_pricing --paths 1000000 --contracts 100 --steps 16 --batch 8192 --seed 20261112 --out reproduced
python3 src/high_precision_reference.py reproduced
```

The C++ program is **src/monte_carlo_pricing.cpp**; the official SDK interface is **src/paceval_main.h** and **src/paceval_main.cpp**; the independent audit is **src/high_precision_reference.py**. The compact publication outputs are in **examples/**: contracts.csv, prices.csv, path_samples.csv, payoff_audit.csv, model_update.csv, tinc_samples.csv, validation.txt and high_precision_audit.json. Every figure is provided as PNG and the eight scientific plots also have editable SVG equivalents in **figures/**.

Environment of the executed reference experiment: Debian Linux 13 on x86-64, AMD EPYC 9V74 (five visible virtual CPUs), GCC 14.2, C++17, Release build, original provided Linux paceval shared runtime; actual study execution is sequential on one process. This is a reproducibility description, not a speed claim.

Selected scientific and technical references

1. paceval, C/C++ API, computation handles, batch functions and TINC: https://paceval.com/api/

2. QuantLib open-source European option example (Monte Carlo and antithetic sampling): https://github.com/lballabio/QuantLib/blob/master/Examples/EquityOption/EquityOption.cpp

3. Black-Scholes formula and risk-neutral geometric Brownian motion: https://wanghemath.github.io/Book-MathFinance/chapters/chapter-16-black-scholes-formula.html

4. Intel oneMKL Cookbook, Black-Scholes pricing: https://www.intel.com/content/www/us/en/docs/onemkl/cookbook/2021-4/black-scholes-formula-european-options-pricing.html

The 100 million output checks and selected independent high-precision audits demonstrate correct reusable payoff evaluation for the specified synthetic workload. They do not constitute independent certification of a financial pricing system, performance superiority or market suitability.